

SmartTaller — Auditoría técnica pre-MVP

Fecha: 4 de julio de 2026 · **Stack:** Next.js 14, Supabase, Tailwind, TypeScript, Server Actions ·

App: apps/smartaller

Resumen ejecutivo

Arquitectura bien encaminada. **2 blockers de seguridad** antes del launch:

1. **Puente por placa sin verificación** — usuario B2C puede ver historial ajeno registrando una placa.
2. **Webhook Telegram con secret opcional** — sin `TELEGRAM_WEBHOOK_SECRET` acepta POST falsos.

1. SEGURIDAD Y RLS

✓ Lo que funciona

- Multi-taller: `taller_id = get_my_taller_id()` en vehículos, mantenimientos, recordatorios
- B2C: CRUD con `user_id = auth.uid()`
- Webhook usa service role solo en servidor
- Middleware protege `/dashboard` y `/app`

● CRÍTICO — Puente por placa

Archivos: `app/actions/vehicles.ts`, `puede_taller.sql`, `process-invoice.ts`

Cualquier usuario en `/app` registra placa ajena → service role asigna `user_id` → RLS expone mantenimientos de esa placa de *cualquier taller*. Chat Smartaller amplifica el leak.

Mitigación MVP: (A) Desactivar política por placa, (B) filtrar por `vehiculo_id`, o (C) aceptar riesgo solo en piloto cerrado.

● CRÍTICO — Webhook sin secret obligatorio

Archivo: `app/api/telegram-webhook/route.ts`

Si falta TELEGRAM_WEBHOOK_SECRET → endpoint abierto → inserts con service role.

Fix: fail-closed en prod si !secret → 503.

● ALTO — RLS legacy using(true)

Verificar en Supabase que no queden políticas `using (true)` para authenticated en mantenimientos, vehiculos, recordatorios.

● ALTO — Token Telegram en URL a OpenAI

getTelegramFileUrl() incluye bot token en URL enviada a OpenAI Vision. Usar downloadTelegramFile + base64.

● MEDIO

- Dos usuarios B2C con misma placa → maybeSingle() falla en webhook
- /api/chat sin rate limit → abuso de créditos OpenAI

2. ROBUSTEZ EN PROD

Flujo: Telegram → waitUntil  → OpenAI → processInvoice → CallMeBot → Telegram reply

Nota: No hay Resend. Solo CallMeBot para WhatsApp.

Riesgo	Detalle
Timeout	maxDuration=60; monitorear en Hobby
Sin idempotencia	Reintentos Telegram = duplicados
Sin transacción	Writes parciales en processInvoice
Bug recordatorios	vehicle-history.ts: query global .limit(1) sin filtro vehículo

3. CLEAN CODE / ANTIGRAVITY

 Server Actions, Zod en createVehicle, useTransition, Server Components, API Route para chat streaming.

 updateNombreTaller sin Zod; dashboard traga errores; sin tipos Supabase generados.

4. ACCIONES INMEDIATAS

P0 — Antes de deploy

1. Obligar TELEGRAM_WEBHOOK_SECRET (fail-closed)
2. Auditar pg_policies en Supabase
3. Acotar puente por placa
4. Filtrar recordatorios por vehiculo_id
5. OpenAI sin URL con bot token

P1 — Robustez

6. UNIQUE (telegram_chat_id, telegram_message_id)
7. Early return idempotente en processInvoice
8. 8 migraciones en orden + env vars Vercel

Variables Vercel (obligatorias)

```
TELEGRAM_BOT_TOKEN
TELEGRAM_WEBHOOK_SECRET
OPENAI_API_KEY
NEXT_PUBLIC_SUPABASE_URL
NEXT_PUBLIC_SUPABASE_ANON_KEY
SUPABASE_URL
SUPABASE_SERVICE_ROLE_KEY
CALLMEBOT_API_KEY (opcional)
```

Checklist deploy

- [] 8 migraciones aplicadas
- [] pg_policies sin using(true)
- [] Webhook con secret_token
- [] Probar /vincular + factura + dashboard
- [] Probar /app + chat Smartaller
- [] Decisión sobre riesgo puente placa

Veredicto: Casi listo para MVP piloto. No para prod abierta sin P0. Tiempo parches P0: ~2-4 h.

SmartTaller · apps/smartaller · monorepo AGENTEIA